Solving a Sudoku from a Photo: Rectification, Grid Detection, Digit CNN and Backtracking

Machine Learning, Course Project

Arshia Goshtasbi • MSc Electrical Engineering (Control), Iran University of Science and Technology •
Spring 1404 (2025)

Goal. Take a phone photo of a printed Sudoku and return the solved puzzle drawn on the image. The project was handed in in four phases; all of them are in `sudoku.ipynb` (and the exported `sudoku.py`).

Phase 1: rectify the paper. Grayscale, Gaussian blur, Canny edges and dilation. The largest external contour is taken as the sheet and approximated by a quadrilateral (or the minimum-area rectangle if the approximation does not have four corners). The corners are ordered and a perspective warp gives a front view of the paper.

Phase 2: find the grid. Adaptive thresholding of the rectified sheet, then the largest convex four-sided contour with an aspect ratio between 0.8 and 1.2.

Phase 3: read the digits. The grid's bounding box is split into 9×9 cells (2-pixel inner padding). A cell whose mean intensity is above 240 counts as empty. Otherwise the largest contour in the Otsu-thresholded cell is cropped, scaled to fit 25×25 and centred on a 28×28 canvas. Two Keras CNNs were trained on the Kaggle *printed digits* dataset (80/20 split):

- a simple model: two convolution and pooling blocks and a 64-unit dense layer;
- an improved model: three convolution blocks with batch normalisation and dropout, global average pooling, and training with augmentation (zoom and shift $\pm 10\%$, contrast $\pm 20\%$), early stopping and learning-rate reduction.

A prediction is only accepted if its softmax confidence is above 0.8.

Phase 4: solve and draw. Depth-first backtracking that checks rows, columns and 3×3 boxes; the missing digits are written on the rectified image in red.

What I re-checked for this repository. The original test photos came with the course and are not included. TensorFlow has no conda-forge build for Windows, so I wrote `keras_numpy.py`, a NumPy forward pass that reads the saved `.h5` models; `run_demo.py` runs the four phases unchanged from `sudoku.py` with it.

- *Classifiers* (`evaluate_models.py`): on all 6299 images of the digits dataset, training images included (the exact Keras split can't be rebuilt without TensorFlow), the simple model scores 98.8% and the improved one 97.4%. The improved model's weakest class is "1" (86.7%).
- *Pipeline* (`make_test_photos.py` renders three synthetic photos with a random perspective, rotation, shading, blur and noise; Figure 1): rectification and cell segmentation work on all three. No digit was misread and no empty cell was read as a digit, but many clues were missed: 14 of 35, 15 of 32 and 1 of 28. In the missed cells a piece of grid line is left inside the crop and is the largest contour, so the classifier sees a line, not the digit. With clues missing, backtracking still finds a valid grid, but not the one that matches the printed puzzle. A tighter crop or removing lines before the contour step would fix this; I left the course code as it was.

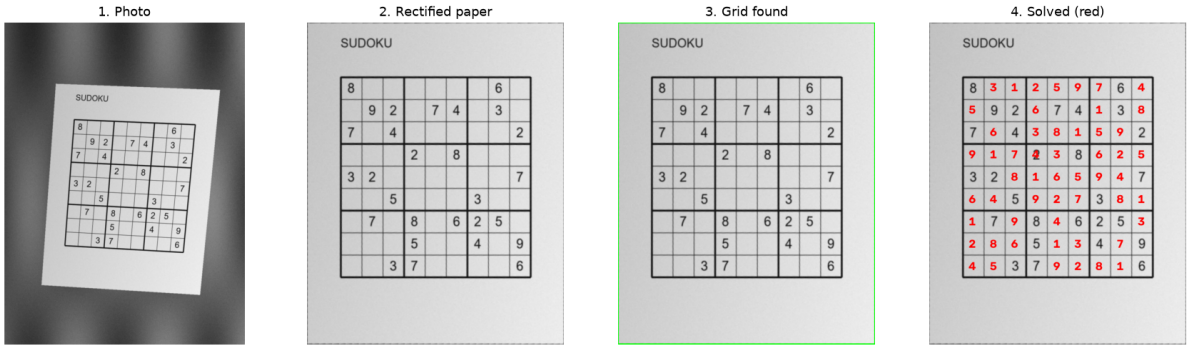


Figure 1: Synthetic photo 3: input, rectified sheet, the Phase 2 outline (here it found the sheet border, not the grid), and the filled-in grid. 27 of 28 clues were read; the 2 in row 4, column 4 was missed, so the completion is a valid grid but not the intended one.